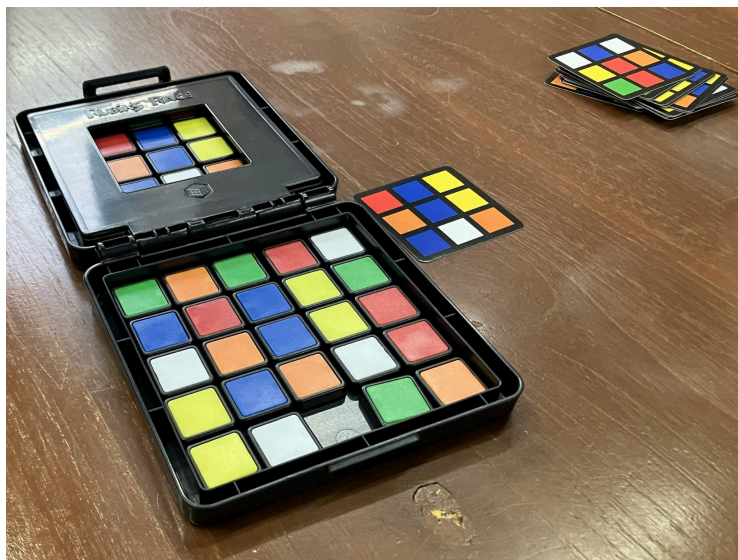


Rubik's Race Optimization

(output only)

Rubik's Race เป็นเกมกระดานปริศนาการเลื่อน (Sliding Puzzle) ที่ต้องอาศัยการเลื่อนแผ่นกระเบื้อง เพื่อจัดเรียงสีให้ได้ตามที่กำหนด โดยมีกฎการเล่นดังต่อไปนี้



(ภาพประกอบ 1)

1. กระดานของผู้เล่น

เป็นตารางสี่เหลี่ยมจัตุรัสขนาด $N \times N$ ช่อง (โดย N เป็นเลขคี่เสมอ เช่น 5, 7 และ 9) ที่ประกอบด้วย

- ช่องว่างขนาด 1×1 จำนวน 1 ช่องเสมอ โดยจะแทนด้วยตัวเลข -1 ซึ่งช่องว่างเป็นตัวเลขที่ทำให้เราสามารถเลื่อนแผ่นกระเบื้องอื่น ๆ ได้
- และที่เหลือจะเป็นแผ่นกระเบื้องขนาด 1×1 โดยมีหมายเลขระบุไว้บนแผ่น หมายเลขดังกล่าวมีค่าได้ตั้งแต่ 1 ถึง 100 และซ้ำกันได้ แทนสีของกระเบื้อง

2. เป้าหมายของเกม

เกมจะกำหนดพื้นที่เป้าหมายขนาด $(N - 2) \times (N - 2)$ ให้ โดยจะอยู่ที่ด้านบนกระดานของผู้เล่น ลักษณะดังภาพประกอบ 1 จงใช้การเลื่อนแผ่นกระเบื้องที่ติดกันกับช่องว่างไปมา จนกว่าพื้นที่เป้าหมาย $(N - 2) \times (N - 2)$ บริเวณกลางกระดานของผู้เล่นจะมีสีเหมือนกันกับพื้นที่เป้าหมายที่กำหนดให้ทุกประการ

(หมายเหตุ: ไม่สนใจสีของแผ่นกระเบื้องที่อยู่วงนอกพื้นที่เป้าหมาย โดยจะเป็นเลขอะไรหรืออยู่สะเปะสะปะแค่ไหนก็ได้)

ตัวอย่างพื้นที่เป้าหมายกลางกระดานของผู้เล่น: หากกระดานมีขนาด 5×5 ($N = 5$) และพื้นที่เป้าหมายขนาด 3×3 พื้นที่เป้าหมายที่ผู้เล่นต้องจัดการจะอยู่ตรงกลางดังภาพ (แทนด้วย o)

x x x x x	<-- ขอบนอก (เป็นเลขอะไรก็ได้)
x o o o x	<--
x o o o x	<-- พื้นที่เป้าหมาย 3x3 ตรงกลาง
x o o o x	<-- (ต้องมีลักษณะเหมือนกับที่กำหนดให้ทุกประการ)
x x x _ x	<-- ขอบนอก (เป็นเลขอะไรก็ได้ หรือเป็นช่องว่างก็ได้)

3. การเคลื่อนที่

ในการเลื่อนแผ่นกระเบื้องไปยังช่องว่าง จะต้องระบุทิศทาง การเลื่อนของแผ่นกระเบื้องที่ติดกับช่องว่าง (-1) ให้เลื่อนเข้ามาแทนที่ช่องว่าง โดยมีคำสั่งแทนลักษณะการเลื่อน 4 ทิศทาง และคำสั่งส่งคำตอบ 1 คำสั่ง ดังนี้:

คำสั่ง	ความหมาย	ผลลัพธ์
U (Up)	สั่งแผ่นกระเบื้องด้านล่าง เลื่อนขึ้นบน	ช่องว่าง -1 จะขยับลงล่าง
D (Down)	สั่งแผ่นกระเบื้องด้านบน เลื่อนลงล่าง	ช่องว่าง -1 จะขยับขึ้นบน
L (Left)	สั่งแผ่นกระเบื้องด้านขวา เลื่อนไปทางซ้าย	ช่องว่าง -1 จะขยับไปทางขวา
R (Right)	สั่งแผ่นกระเบื้องด้านซ้าย เลื่อนไปทางขวา	ช่องว่าง -1 จะขยับไปทางซ้าย
S (Submit)	ยืนยันการส่งคำตอบ	ยืนยันเมื่อพื้นที่เป้าหมายมีลักษณะ เหมือนกับที่กำหนดแล้ว (คำสั่งนี้จะไม่นับเป็น Move ในการคิดคะแนน)

(หมายเหตุ: ไม่สามารถสั่งให้แผ่นกระเบื้องที่อยู่ชิดขอบกระดานเลื่อนทะลุกระดานได้)

ในโจทย์ข้อนี้ คุณจะต้องเขียนโปรแกรมเพื่อหา ลำดับการเลื่อนแผ่นกระเบื้องที่ทำให้สับนพื้นที่เป้าหมายถูกจัดเรียงตามที่กำหนด โดยใช้จำนวนครั้งในการเลื่อนน้อยที่สุด

ข้อมูลนำเข้า

- **บรรทัดที่ 1:** จำนวนเต็ม N ระบุขนาดของกระดาน (N เป็นเลขคู่เสมอ และ $5 \leq N \leq 67$)
- **บรรทัดที่ 2 ถึง $N + 1$:** ตารางขนาด $N \times N$ แสดงสถานะเริ่มต้นของกระดาน แต่ละบรรทัดประกอบด้วยจำนวนเต็มอยู่ N จำนวน คั่นด้วยช่องว่าง โดยแผ่นกระเบื้องปกติจะเป็นตัวเลข 1 ถึง $N + 1$ กระเบื้องแต่ละสีจะมี $N - 1$ ชิ้นพอดี และจะมีตัวเลข -1 อยู่เพียง 1 ตัวเท่านั้นเพื่อแทนช่องว่าง
- **บรรทัดที่ $N + 2$ ถึง $2N - 1$:** พื้นที่เป้าหมายที่กำหนดให้ขนาด $(N - 2) \times (N - 2)$ แต่ละบรรทัดประกอบด้วยจำนวนเต็มอยู่ $N - 2$ จำนวน คั่นด้วยช่องว่าง

ข้อมูลส่งออก

- **บรรทัดเดียว:** สตริงของตัวอักษร U, D, L, R ต่อเนื่องกัน แทนลำดับการเลื่อนของกระเบื้อง และปิดท้ายด้วยตัวอักษร S เสมอ เพื่อยืนยันคำตอบ (ตัวพิมพ์ใหญ่ทั้งหมด **ไม่มีช่องว่าง**)
- หากกระดานเริ่มต้นมีพื้นที่เป้าหมายตรงกันกับที่กำหนดให้อยู่แล้ว สามารถตอบ S ได้เลยโดยไม่ต้องเลื่อนกระเบื้อง

เกณฑ์การให้คะแนน

โจทย์ข้อนี้มีการให้คะแนนแบบ **Partial Scoring** ตามประสิทธิภาพของคำตอบ

1. ถ้าคำตอบทำให้พื้นที่เป้าหมายมีลักษณะไม่ตรงตามที่กำหนด มีการเคลื่อนที่ออกนอกขอบกระดาน หรือมีการเคลื่อนที่ที่ไม่ทำให้แผ่นกระเบื้องขยับ จะไม่ได้รับคะแนน
2. ถ้าคำตอบถูกต้อง คะแนนจะคำนวณจากจำนวนครั้งที่ใช้เลื่อนของคุณ (L) เทียบกับจำนวนครั้งที่ solution ของระบบใช้ (Baseline - $BASE$)
 - ถ้า $L \leq BASE$: จะได้ 80% ของคะแนนในกรณีนั้น
 - ถ้า $L > BASE$: จะได้ $80(\frac{BASE}{L})$ % ของคะแนนในกรณีนั้น
3. อีก 30% จะได้ตามอันดับ **เทียบกับนิสิตทุกคนในคลาส** ของจำนวนครั้งที่ใช้ในการเลื่อน (น้อยกว่าถือว่าดีกว่า **ได้คะแนน linear ตามอันดับ จาก 30% ไปจนถึง 0%**)

นั่นหมายความว่า นิสิตสามารถได้คะแนนรวมสูงสุดถึง 110% หาก Ranking อยู่อันดับต้นๆ ของคลาส

ตัวอย่างการทำงานของโปรแกรม

ข้อมูลนำเข้า	ข้อมูลส่งออกแบบหนึ่งที่เป็นไปได้
5 1 2 3 4 5 2 6 1 3 4 -1 6 3 2 5 5 4 1 2 1 6 5 4 3 6 6 1 5 6 3 2 4 1 2	DDLILLURDS